

MeteoAgent-DA 面试深挖与八股手册

三天速成版见：[docs/three_day_interview_stack.md](#)。

这份文档的目标不是背稿，而是让你在面试中能把项目讲成一个完整闭环：

卫星资料同化科研工作流
-> 工具增强 Agent
-> trace / report / artifacts
-> verifier / reward
-> SFT / DPO / GRPO-ready post-training data
-> PASBench-DA before-after evaluation

最重要的边界：当前项目已经实现后训练数据管线、偏好数据构造、reward breakdown、verifier 和 tiny LoRA dry-run 入口；除非你确实跑完训练和评测，否则不要说“已经完成 GRPO 训练”或“模型指标真实提升”。更专业的说法是：

我构建了一个面向资料同化科研 Agent 的 post-training data pipeline，支持成功轨迹过滤、SFT 数据构造、DPO preference pair、GRPO-ready reward 信号和 PASBench-DA 可执行评测。

30 秒开场

MeteoAgent-DA 是一个面向卫星资料同化科研工作流的工具增强 Agent 和后训练数据管线。它不是泛天气问答助手，而是围绕 FY-3F MWTS-III、ERA5 和 PASNet-DA 温度廓线订正，把数据检查、训练命令生成、指标解析、绘图和论文表格生成封装成可验证工具。Agent 每次运行都会落 `trace.jsonl`、`report.json` 和 artifacts，然后用 verifier 过滤成功轨迹，构造 SFT 数据、DPO 偏好数据和 GRPO-ready reward breakdown，最后用 PASBench-DA 评估工具调用成功率、命令可复现性、artifact 生成和幻觉率。

2 分钟版本

我做这个项目时刻意避开了“泛天气聊天机器人”的方向，因为那种项目很难证明模型真的会做科研工作流。我把任务收敛到一个真实主线：FY-3F MWTS-III 微波亮温观测和 ERA5 背景场，用 PASNet-DA 做温度廓线增量订正。

系统分成四层。第一层是 Agent Runtime，负责 planning、tool call、reflection、report 和 trace 落盘。第二层是领域工具，包括 `sanity_check`、`data_indexer`、`pasnet_runner`、`evaluator`、`plotter`、`paper_writer`。第三层是 PASBench-DA，评测数据检查、命令生成、结果解释和论文写作。第四层是后训练管线：`filter_traces.py` 过滤失败或不可验证轨迹，`build_sft_data.py` 构造 SFT messages，`build_preference_data.py` 构造四类 DPO pair，`rewards.py` 输出可解释 reward breakdown，`build_rollout_rewards.py` 导出 rollout/reward JSONL。

这个项目的核心价值不只是“我写了一个 Agent”，而是我知道 Agent 后训练需要什么数据、如何清洗、如何构造 preference、如何设计 verifier/reward，以及如何用 benchmark 验证模型是否真的变好。

面试展开路线

0-5 分钟：先给定位

面试官通常先问“介绍一下项目”。不要从代码文件开始讲，先讲问题定义：

- 资料同化科研任务很强调路径、数据划分、命令参数、指标文件和论文产物。
- 纯文本 LLM 容易给出 plausible 但不可验证的回答。
- 所以我把任务做成 tool-grounded workflow，并沉淀成后训练数据。

一句话：

这个项目的第一目标是让 LLM 在气象资料同化科研中可执行、可验证、可复盘；第二目标是把这些可执行轨迹转成后训练数据。

5-15 分钟：讲系统架构

按这个顺序讲：

1. `agent/runtime.py`：反思式主循环，生成 plan，逐步调用工具，记录 trace，生成 report。
2. `agent/schemas.py`：AgentTask、ToolCall、ToolResult、AgentReport，保证中间产物结构化。
3. `tools/registry.py`：显式工具注册表，LLM 不能随便发明工具。
4. `tools/pasnet_runner.py`：生成受控训练命令，默认 dry-run。
5. `bench/pasbench.py` 和 `bench/verifiers.py`：把报告转成可量化指标。
6. `post_training/` 和 `verifiers/`：后训练数据和质量门控。

白板图：

```
User Task
-> Planner
-> ToolRegistry
-> Domain Tool
-> ToolResult
-> Reflection
-> AgentReport
-> Verifier
-> SFT / DPO / Reward Data
```

15-25 分钟：讲后训练

这里要明显像后训练岗位，而不是普通应用岗位。

SFT：

- 输入：通过 verifier 的成功 trace。
- 输出：system/user/assistant messages。
- assistant 不是普通自然语言，而是结构化 plan、tool trajectory、observations、reflections、final summary。
- 目的：让模型学习“怎么规划、怎么调用工具、怎么根据 observation 修正”。

DPO：

- 输入：chosen/rejected report。
- 四类偏好：
- `tool_grounded_vs_text_only`
- `verified_report_vs_unverified_report`
- `successful_recovery_vs_failed_recovery`
- `concise_reproducible_vs_verbose_hallucinated`
- 目的：不只偏好“看起来更好”的答案，而是偏好可执行、可验证、少幻觉的 Agent 行为。

GRPO-ready reward：

- 当前不声称完成 GRPO。
- 已有 reward breakdown：
- tool success
- artifact completeness

- scientific consistency
- format compliance
- reproducibility
- hallucination penalty
- unsafe command penalty
- 目的：让后续 RL-style optimization 有可解释 reward signal。

25-35 分钟：讲评测

PASBench-DA 评测的不是“气象知识问答”，而是科研工作流能力：

- data query
- experiment planning
- result reasoning
- paper writing
- failure recovery
- negative/OOD tasks

核心指标：

- **TSR**: task success rate。
- **VER**: verifier pass rate。
- **TCR**: tool-call recall。
- **CVR**: command validity rate。
- **AGR**: artifact generation rate。
- **HAL**: hallucinated artifact/metric/path rate，越低越好。
- **Cost**: 工具步数或运行成本，越低越好。

强调一句：

我没有只看最终文本，而是检查 tool observation、命令参数、路径、metric artifact、报告中的 evidence 和 dry-run 诚实性。

35-45 分钟：应对深挖

面试官会挑刺：

- 你这个是不是规则系统？
- 没训练模型为什么叫后训练项目？
- DPO rejected 怎么保证不是太弱？
- Reward 会不会被 hack？
- Verifier 错了怎么办？
- 为什么不用 RAG？
- 为什么不用直接让 LLM 写 shell？
- 气象同化部分你到底懂多少？

回答时的核心姿态：

- 承认边界。

- 解释工程取舍。
- 把项目拉回“可执行、可验证、可训练数据”。

项目文件速查

```
| 文件 | 面试讲法 |
|---|---|
| `meteo_agent_da/agent/runtime.py` | Agent loop, 负责 plan -&gt; tool -&gt; reflect -&gt; report, 并写 trace |
| `meteo_agent_da/agent/schemas.py` | 结构化数据协议, 避免纯文本中间态 |
| `meteo_agent_da/tools/registry.py` | 显式工具注册, 控制可用工具集合 |
| `meteo_agent_da/tools/pasnet_runner.py` | 构造 PASNet-DA dry-run 命令, 保证参数完整 |
| `meteo_agent_da/bench/pasbench.py` | PASBench-DA task loader、score 和 aggregate |
| `meteo_agent_da/bench/verifiers.py` | benchmark 侧 verifier, 用于评测表 |
| `meteo_agent_da/verifiers/` | post-training 侧 verifier, 用于数据过滤 |
| `meteo_agent_da/post_training/filter_traces.py` | 轨迹过滤, 拒绝失败、幻觉、不可复现样本 |
| `meteo_agent_da/post_training/build_sft_data.py` | 成功 trace -&gt; SFT messages |
| `meteo_agent_da/post_training/build_preference_data.py` | chosen/rejected -&gt; DPO preference |
| `meteo_agent_da/post_training/rewards.py` | 可解释 reward breakdown |
| `meteo_agent_da/post_training/build_rollout_rewards.py` | PASBench scores -&gt; rollout/reward JSONL |
| `examples/preference_cases/` | 四类偏好样例 |
| `examples/failure_traces/` | 错误-反思-修复样例 |
| `docs/interview_playbook.md` | 当前面试手册 |
```

后训练八股

SFT 是什么，为什么需要？

SFT 是监督微调，用高质量 instruction/response 数据让模型学习目标行为。在 Agent 场景中，response 不应该只有最终答案，还应该包含规划、工具调用参数、工具返回观察、反思和最终报告。MeteoAgent-DA 的 SFT 数据来自成功工具轨迹，所以学习目标是“完成科研工作流”，不是背气象常识。

可能追问：SFT 的风险？

- 会学习到错误轨迹，所以要先过滤。
- 可能过拟合固定工具顺序，所以 benchmark 要有多任务和 near-domain/OOD。
- 可能让模型变啰嗦，所以 DPO 可以偏好 concise/reproducible。

DPO 是什么，为什么不用 PPO？

DPO 直接用 preference pair 优化策略，让 chosen 相对 rejected 的概率更高，不需要在线训练 reward model，也不需要 PPO 的复杂采样和 value function。对这个项目来说，先有 chosen/rejected report 更现实：tool-grounded > text-only, verified > unverified, recovery > failure。

可能追问：DPO 数据怎么构造？

- 自动：从 PASBench scores 中选择 `heuristic_tool` 成功报告作为 chosen，`text_only` 或失败报告作为 rejected。
- 人工/半自动：补充四类 preference cases。
- 质量控制：metadata 记录 `preference_type`、chosen/rejected report、verifier 结果。

GRPO 是什么，你做了吗？

GRPO 是一种 RL-style group relative optimization 思路，常见于让模型在多个候选输出之间基于相对 reward 学习。这个项目目前做的是 GRPO-ready reward design：把 rollout 和 reward breakdown 导出为 JSONL，为后续 GRPO/RLAIF 做数据准备。不要说已经完成 GRPO 训练。

安全表述：

我目前完成的是 reward signal 和 rollout 数据导出，后续可以接 GRPO trainer。这样说比直接声称跑过 GRPO 更诚实。

Reward breakdown 为什么重要？

单个 reward 分数不可解释，面试官会担心 reward hacking。拆分后可以看到：

- 工具是否成功；
- artifact 是否存在；
- 科学检查是否通过；
- 命令是否可复现；
- 是否有路径/指标幻觉；
- 是否用了不安全命令。

这让错误分析更容易，也能为 DPO/RL 样本清洗提供依据。

Trajectory filtering 为什么重要？

Agent 后训练最怕把错误行为蒸馏进模型。过滤规则包括：

- 工具失败；
- 无工具证据；
- artifact 缺失；
- 命令路径不可复现；
- dry-run 下虚假宣称训练完成；
- 指标无 evaluator 证据；
- trace 过长、重复、JSON 错误。

一句强回答：

我不是把所有 Agent 输出都灌进 SFT，而是只保留可执行、可验证、可复盘的成功轨迹，避免把幻觉和坏工具调用放大。

Reward hacking 怎么办？

可能问题：模型会不会为了拿高 artifact_reward 随便写假路径？

回答：

- artifact verifier 不只看 report 声明，还检查路径是否存在。
- command verifier 检查关键参数和 split_file。
- scientific verifier 检查 dry-run 虚假训练声明和无来源指标。
- PASBench 指标中 **HAL** 专门惩罚 hallucinated path/metric/artifact。
- 后续可以引入人工 spot-check 和 adversarial benchmark。

Preference pair 会不会太弱？

如果 rejected 永远是 text-only，DPO 会学到“只要有工具就行”，不一定学会更细的质量偏好。所以项目里把 pair 分成四类：

- 工具证据 vs 纯文本；
- verifier-clean vs unverified；

- 错误恢复 vs 失败停住；
- 简洁可复现 vs 冗长幻觉。

这样 rejected 难度逐步提高，更贴近真实 Agent 后训练。

LoRA/QLoRA 怎么讲？

LoRA 是低秩适配，在冻结大模型主参数的情况下训练少量低秩矩阵，降低显存和训练成本。关键超参：

- r ：低秩维度，越大容量越强但更容易过拟合。
- α ：缩放系数。
- dropout：防止小数据过拟合。
- target modules：通常是 attention projection 和 MLP projection。

QLoRA 进一步把基座模型量化到 4-bit，再训练 LoRA adapter，适合单卡或小显存。

Agent 八股

ReAct 是什么？

ReAct 是 reasoning + acting：模型先思考，再调用工具，根据 observation 继续思考。MeteoAgent-DA 的 runtime 对应：

```
plan -&gt; tool -&gt; observation -&gt; reflection -&gt; report
```

区别是这里把每一步结构化落盘，方便评测和后训练。

Tool calling 的关键问题？

- 工具 schema 要明确。
- 工具名不能幻觉。
- 参数要可验证。
- 工具结果要结构化。
- 执行权限要受控。
- 失败要能反思恢复。
- trace 要能复盘。

为什么不用 LLM 直接执行 shell？

因为科研环境里路径、GPU、训练脚本和数据很容易出错，直接 shell 风险高。项目用 ToolRegistry 和 ProjectConfig 限定工具和参数，pasnet_runner 默认 dry-run，只生成命令，不直接执行昂贵训练。这样能把安全性和可复现性放在第一位。

Memory 有什么用？

会话记忆可以记录：

- 已检查过的数据路径；
- 当前 split；
- 上一次生成的命令；
- 缺失 artifact；

- 用户偏好的模型或区域。

但 memory 不能替代 verifier。记忆是上下文，verifier 才是证据。

Reflection 有什么用？

Reflection 不是为了“自言自语”，而是把失败原因显式记录下来，例如 split 文件不存在、metric artifact 缺失、dry-run 不能声称训练完成。后训练时，失败-反思-修复轨迹能训练模型的 recovery 能力。

Benchmark 为什么不能只看 BLEU/ROUGE？

Agent 的目标不是生成和参考答案相似的文本，而是完成任务。这里更重要的是：

- 工具是否调用对；
- 命令是否可执行；
- artifact 是否真实存在；
- 指标是否有来源；
- 是否少幻觉；
- 成本是否可接受。

资料同化八股

资料同化是什么？

资料同化是把观测资料和数值模式背景场结合，得到更接近真实大气状态的分析场。经典形式：

$$\mathbf{x}_a = \mathbf{x}_b + \text{increment}$$

其中 $\mathbf{x}_b$ 是背景场， $\mathbf{x}_a$ 是分析场，increment 是对背景的订正。

在这个项目里，学习问题被简化成离线增量订正：

PASNet input: FY-3F brightness temperatures + observation mask + ERA5 background
target: ERA5 analysis/reference increment

O-B 和 O-A 是什么？

- O-B：observation minus background，观测和背景的差。
- O-A：observation minus analysis，观测和分析的差。

如果同化有效，通常希望 O-A 的统计误差小于 O-B，但要注意观测误差、代表性误差和独立验证。

3DVar / 4DVar / EnKF 区别？

3DVar：在单一分析时刻最小化代价函数：

$$J(\mathbf{x}) = (\mathbf{x} - \mathbf{x}_b)^T \mathbf{B}^{-1} (\mathbf{x} - \mathbf{x}_b) + (\mathbf{y} - \mathbf{H}\mathbf{x})^T \mathbf{R}^{-1} (\mathbf{y} - \mathbf{H}\mathbf{x})$$

4DVar：在时间窗口内考虑模式演变，对初始状态求最优。

EnKF：用集合预报估计背景误差协方差，通过 Kalman update 融合观测。

项目里的 PASNet-DA 不是经典变分同化系统，而是学习一个 offline increment correction，可以作为 ML-DA / neural correction 的工作流。

亮温是什么？

微波亮温是卫星辐射计观测到的辐射强度换算成等效黑体温度。MWTS-III 的氧气吸收带通道对大气温度廓线敏感，窗口通道对地表和云雨等更敏感。模型用 17 通道亮温和 mask 提供观测信息。

为什么要 observation mask？

卫星观测是稀疏且不规则的。栅格化后有些格点没有观测。如果直接用 0 填充，模型可能把“没有观测”误解成真实亮温值。mask 显式告诉模型哪些格点有观测。

为什么要 train split 统计量？

归一化统计量必须只从训练集计算，避免验证集和测试集信息泄漏。这个点在科研和面试里都很关键。

RMSE / MAE / ACC 怎么解释？

- RMSE：平方误差均值开根号，对大误差更敏感。
- MAE：绝对误差均值，更稳健。
- ACC：异常相关系数，常用于天气预报场相似性评估。

如果没有 evaluator 解析到 metric 文件，不要声称具体数值。

LLM 八股

Transformer 核心是什么？

自注意力机制根据 query/key/value 计算 token 间依赖：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) V$$

多头注意力让模型在不同子空间关注不同关系。

RoPE 是什么？

RoPE 是旋转位置编码，把位置信息注入 query/key，常用于现代 LLM，支持较好的相对位置建模和长度外推。

KV cache 是什么？

推理时缓存历史 token 的 key/value，生成下一个 token 时不用重复计算全部前文，能显著加速自回归解码。

Temperature / top-p 怎么影响输出？

- temperature 越高越随机。
- top-p 只从累计概率达到 p 的候选 token 中采样。
- Agent 工具调用通常要低 temperature，提高稳定性。

RAG 和 Tool Agent 区别？

RAG 主要检索文本知识，Tool Agent 调用可执行工具。这个项目需要检查路径、生成命令、解析 metric、写 artifact，所以 tool grounding 比单纯 RAG 更核心。后续可以把领域文档作为 RAG 补充，但不能替代工具 verifier。

常见盘问与回答

Q1：你这个项目和普通气象问答有什么区别？

普通气象问答重在知识回答，MeteoAgent-DA 重在可执行科研工作流。它会检查数据路径、split、stats、生成训练命令、解析指标和生成论文 artifact，并把过程落成 trace 用于后训练。

Q2：你这个是不是把规则写死了？

第一版工具和 verifier 是规则化的，这是为了保证可控和可评测。真正要训练的是 planner/chat LLM：它在固定工具契约下学习何时调用哪个工具、如何解释 observation、如何从失败恢复。工具规则提供 environment 和 reward，不是最终模型能力的全部。

Q3：为什么需要 benchmark？

没有 benchmark 就只能展示 demo，无法证明工具 Agent 比 text-only 好。PASBench-DA 能量化工具召回、命令有效性、artifact、verifier pass、幻觉率和成本。

Q4：为什么需要 text-only baseline？

它是最低基线，用来证明纯文本回答虽然流畅，但无法验证路径、命令和 artifact。DPO 中它也可以作为早期 rejected。

Q5：为什么要区分 benchmark verifier 和 post-training verifier？

benchmark verifier 关注任务是否解决，post-training verifier 关注样本是否适合进入训练集。前者服务评测，后者服务数据清洗，两者指标相似但用途不同。

Q6：怎么避免数据泄漏？

归一化统计只用训练 split；PASBench 的任务和测试报告分开；后训练样本来自训练任务或成功轨迹，评测时要使用 held-out PASBench 任务。metric 声明必须来自 evaluator artifact。

Q7：如果 verifier 错误怎么办？

Verifier 是质量门控，不是绝对真理。应对方式：

- 设计多维 verifier，降低单点误判。
- 保留 failed_checks 便于审计。
- 对 reward 边界样本做人工抽查。
- 增加 adversarial cases 更新 verifier。

Q8：为什么不用端到端神经网络直接输出论文？

论文结果必须来自数据和实验。端到端文本生成会放大幻觉。这个项目把实验证据、metric artifact 和 LaTeX 产物作为工具输出，报告只解释证据。

Q9：为什么 dry-run 也有价值？

dry-run 不能证明模型性能，但能验证命令构造、路径、参数、split 和输出目录，降低真实训练前的错误成本。项目明确区分 dry-run 和 executed run。

Q10：你会怎么做下一步？

优先级：

1. 跑完整 PASBench before-after 表。
2. 用 Qwen planner 采集更多成功轨迹。
3. 扩充 preference pair，减少过弱 rejected。
4. 用 LoRA 做小规模 SFT，再评估工具调用成功率。
5. 在 held-out PASBench 上比较 SFT/DPO 前后。

Q11：DPO pair 的 chosen/rejected 怎么保证客观？

不是只靠人工感觉，而是结合 verifier 指标和报告元数据。chosen 应有工具证据、artifact、命令参数和科学一致性；rejected 则缺工具、缺 artifact、失败未恢复或出现幻觉。

Q12：如果模型学会为了 reward 多调用工具怎么办？

指标里有 cost 和 repeated tool calls，PASBench 设置 max_tool_steps。Reward 也应该鼓励必要工具而不是工具越多越好。

Q13：为什么要保存 trace，而不是只保存最终 report？

最终 report 不能反映中间决策。Trace 包含 plan、tool call、observation、reflection，能训练 Agent 的过程能力，也能定位错误来源。

Q14：项目里最核心的工程取舍是什么？

把 LLM 的自由度限制在 tool contract 内，把不可控的自然语言输出变成结构化 report 和可执行 evidence。这样牺牲了一些开放性，但换来了可验证和可训练。

Q15：你的 reward 权重怎么来的？

当前是启发式权重，基于任务重要性设定：verifier/tool/artifact/scientific/reproducibility/penalty。它不是最终真理，后续可以通过人工偏好、ablation 或 grid search 调整。

Q16：怎么证明 SFT 后真的变好？

用 held-out PASBench：

- base Qwen；
- Qwen tool agent；
- SFT Qwen；
- SFT + DPO Qwen。

比较 TSR、VER、TCR、CVR、AGR、HAL、Cost，且必须固定任务、attempts、工具环境和 dry-run 配置。

Q17：为什么选择 FY-3F MWTS-III 和 ERA5？

它们构成了清晰的卫星温度探测和再分析背景场场景，适合构造离线增量订正任务。MWTS-III 提供温度敏感通道，ERA5 提供背景和 reference，PASNet-DA 是主线模型。

Q18：PASNet-DA 和传统同化是什么关系？

PASNet-DA 不是完整 NWP 同化系统，而是学习背景场到参考场的离线增量订正。它可以看作 ML-assisted DA workflow 的一个子问题。

Q19：为什么要保留 land mask，而 full-GCE 只做诊断？

为了控制输入变量和消融解释。默认只使用 land mask，避免 latitude/longitude/solar zenith angle 等上下文变量带来混杂；full-GCE 作为诊断变体检查上下文增益。

Q20：如果面试官让你现场 demo？

可以跑：

```
python -m unittest discover -s tests
python -m meteo_agent_da.cli --task "Compare PASNet and Swin-UNet on the 50pct split and generate a paper table" --dry-run
python -m meteo_agent_da.bench.run_pasbench --tasks examples/pasbench_da_sample.jsonl --method text_only heuristic_tool --output-dir runs/interview_demo
python -m meteo_agent_da.post_training.build_preference_data --scores-json runs/interview_demo/scores.json --output runs/interview_demo/preferences.jsonl
```

代码深挖准备

Runtime

可能问：`trace()` 为什么每次打开文件数行？

当前实现简单可靠，适合小规模本地 trace。大规模运行可以把 step counter 放内存中，或使用 structured logger。

ToolRegistry

可能问：如何防止未知工具？

`ToolRegistry.run()` 查不到工具就返回 `unknown_tool`，不会执行任意命令。`risky=True` 的工具在非 dry-run 下还要求 `allow_risky=true`。

pasnet_runner

可能问：为什么只生成命令不执行？

训练昂贵且依赖 GPU/路径，默认 dry-run 更安全。真实执行前需要检查 GPU、输出目录、split 和 stats 文件。

bench/verifiers.py

可能问：评测为什么看 report 而不是重新执行？

报告中包含 tool observations 和 artifact paths，可以进行离线审计。对于高风险命令，重新执行成本高，先做静态 verifier；后续可为轻量工具增加 replay verifier。

post_training/filter_traces.py

可能问：为什么默认要求 artifacts？

论文写作和实验分析任务需要 artifacts；数据检查任务可能没有 artifact，所以脚本提供 `--allow-no-artifacts`。这是按任务类型调整的过滤策略。

简历 bullet

可以写：

- 构建面向 FY-3F/ERA5/PASNet-DA 科研工作流的 Tool-use Agent Harness，支持任务规划、工具调用、受控执行、trace 落盘与 artifacts 验证。
- 设计 Agent 后训练数据管线，将成功执行轨迹转换为 SFT messages，并基于 tool-grounded/text-only、verified/unverified、recovery/failure 等对比构造 DPO preference pairs。
- 构建 executable workflow reward 与 verifier 体系，从工具成功率、artifact 完整性、科学检查、格式合规、可复现性和幻觉风险等维度评估 Agent 行为。
- 设计 PASBench-DA 评测集，对数据检查、命令生成、实验分析、论文表格生成和失败恢复能力进行量化评测，形成 SFT/DPO 前后对比接口。

不要踩的坑

- 不要说已经完成 GRPO 训练，除非真的跑了。
- 不要说 dry-run 证明模型性能提升。
- 不要声称 FY-3F/ERA5 实验指标，除非 evaluator 找到了对应 metric artifact。
- 不要把项目讲成“气象聊天机器人”。
- 不要过度强调 prompt，重点是 tool contract、verifier、benchmark 和 post-training data。

最后一段收束

如果面试快结束，可以这样总结：

这个项目的重点不是做一个炫技 demo，而是把一个真实资料同化科研流程变成 Agent 可以执行、系统可以验证、模型可以后训练的数据闭环。它覆盖了工具调用、trace 采集、trajectory filtering、SFT/DPO 数据构造、reward breakdown 和 benchmark evaluation，这些正是 Agent 后训练岗位最关心的链路。